목표: 데이터를 리스트에 저장하고 가중치 연산 및 등급을 판정합니다.
이렇게 짰어 문제 있는지 봐줘

작성해주신 **[V1.0] 코드는 아주 깔끔하고 기능적으로 완벽합니다!** 파이썬의 기초 연산과 포맷팅(f-string)을 정확하게 활용하고 계시네요.

굳이 개선할 점을 하나 꼽자면, 나중에 입력값을 테스트할 때 **'매수 가격'이나 '수량'에 0을 입력하면** `ZeroDivisionError` **가 발생**할 수 있다는 점입니다. (물론 과제 제출용으로는 이 정도로도 충분합니다!)

자, 이제 **[V2.0: 리스트와 조건문]** 단계로 넘어가기 위한 가이드를 드릴게요. 2차 과제 목표에 맞춰서 어떻게 코드를 확장하면 좋을지 방향성을 잡아보겠습니다.

💡 [V2.0] 개발 가이드: 리스트와 조건문 활용

2차 과제에서는 **"한 번의 매매"가 아니라 "여러 번의 매매를 기록하고 분석하는 것"**이 핵심입니다.

1. 데이터 구조 개선 (리스트)

이제 변수를 개별로 두지 말고, **'리스트(list)'**를 사용하여 여러 매매 기록을 담아야 합니다.

- `trade_history = []` 와 같은 리스트를 만들고, 매매가 발생할 때마다 `trade_history.append()` 를 통해 정보를 저장하세요.

2. 등급 판정 (조건문 if-elif-else)

수익률(`profit_rate`)을 기준으로 등급을 나누는 `if` 문을 추가하세요. 퀀트 관점에서 다음과 같은 로직을 추천합니다.

- **S등급:** 수익률 20% 이상
- **A등급:** 수익률 10% 이상 ~ 20% 미만
- **B등급:** 0% 이상 ~ 10% 미만
- **F등급:** 0% 미만 (손실)

2차 과제 제출을 위한 문서 초안을 작성해 드립니다. 기존의 '용사' 예시를 **질문자님의 'My Quant Log' 컨셉과 과제 요구사항에 딱 맞춰서** 수정했습니다.

이 내용을 복사해서 과제 제출 파일(보고서)에 그대로 활용하세요!

🟩 [2차 과제: V2.0] 데이터 구조화 및 성과 진단 시스템 구현

✨ 2차 과제 내용

이번 과제는 단순한 매매 기록 입력을 넘어, 파이썬의 핵심 자료구조인 **리스트(List)**를 활용해 데이터를 축적하고, **조건문(if-elif-else)**을 이용해 전략 성과를 자동으로 진단하는 시스템을 구축하는 것입니다. 리스트 조작 메서드(`append` , `insert`)를 사용하여 실시간으로 데이터를 관리하고, 슬라이싱(`[:]`)을 통해 원본 데이터 보호와 분석용 복사본 생성을 구현하였습니다. 최종적으로 수익률을 기반으로 S등급부터 F등급까지 판정하는 자동화 로직을 완성하였습니다.

- **핵심 기능:**
  1. 개별 변수 대신 2차원 리스트를 사용하여 매매 데이터(종목, 매수가, 매도가, 수량)를 체계적으로 관리.
  2. `append()` 로 기록 추가, `insert(0, ...)` 로 중요 지수/이벤트 매매 최상단 삽입.
  3. `[:]` 연산자를 활용한 리스트 복사로 원본 데이터 유지 및 분석 데이터 분리.
  4. `if-elif-else` 를 사용한 5단계 매매 등급 판정 로직 구현.

🤖 AI 파트너십 과정

- **내용 1: 데이터 구조화 및 퀀트 전략 로직 도출**
  - **프롬프트 요약:** "1차 과제에서 만든 매매 일지를 리스트 기반으로 바꾸고 싶어. 수익률을 계산해서 등급을 매기는 로직을 파이썬 조건문으로 깔끔하게 짜줄 수 있어?"
  - **적용 내용:** AI가 제안한 '2차원 리스트(List of Lists)' 구조를 채택하여 매매 데이터를 관리함. 단순히 숫자를 출력하는 것에서 나아가, `if-elif-else` 를 사용하여 수익률 구간별로 'S등급(슈퍼 트레이더)'부터 'F등급(손실 주의)'까지 나누는 퀀트적 진단 로직을 성공적으로 설계함.
- **내용 2: 리스트 조작 기술 및 메모리 보호 학습**
  - **프롬프트 요약:** "리스트에 데이터를 추가할 때 그냥 추가하는 것(`append`) 말고 맨 앞에 넣는 법(`insert`)도 알려줘. 그리고 리스트를 복사할 때 원본이 바뀌지 않게 하려면 어떻게 해야 해?"
  - **적용 내용:** 데이터 관리의 전문성을 높이기 위해 `append` 와 `insert` 의 차이를 학습하여 실제

칭호: 시장의 지배자 이렇게 s등급은 칭호도 주고 했는데 왜 나머지는 없어?

정확한 지적입니다! 1차 코드를 확장하면서 핵심 로직을 보여드리는 데 집중하다 보니, 다른 등급의 칭호는 생략했었네요.

과제 요구사항에 "and/or 논리 연산자를 활용하여 상황에 맞는 칭호를 부여하라"는 내용이 있으니, **모든 등급에 칭호를 부여하고, 추가적인 조건을 걸어 더 풍성한 로직으로 수정**해보겠습니다. 교수님께 "조건을 복합적으로 걸어 로직을 고도화했다"고 어필하기 딱 좋습니다.

💡 수정된 코드: 모든 등급별 칭호 및 복합 조건(and/or) 로직

이 코드는 단순히 등급만 매기는 게 아니라, '투자 금액(amount * buy_price)'이라는 변수까지 함께 고려하여 `and` 연산자로 칭호를 부여합니다. 이렇게 하면 교수님께서 원하시는 "지능형 로직"의 느낌을 확실히 줄 수 있습니다.

Python

```python
# [V2.0] 성과 분석 및 칭호 부여 시스템
print("\n--- [성과 분석 리포트] ---")
analysis_list = trade_log[:]

for trade in analysis_list:
    name, buy, sell, amt = trade
    profit_rate = ((sell - buy) / buy) * 100
    invest_amount = buy * amt # 투자 금액 계산 (복합 조건용)

    # 등급 판정 및 칭호 부여 (and 연산자 적극 활용)
    if profit_rate >= 20 and invest_amount >= 100000:
        grade = "S"
        title = "시장의 지배자 (고액 거물)"
    elif profit_rate >= 20:
        grade = "S"
        title = "전설의 트레이더"
    elif profit_rate >= 10:
        grade = "A"
        title = "전문 트레이더"
    elif profit_rate >= 0:
        grade = "B"
        title = "안정적 트레이더"
```

invest_amount = buy * amt # 투자 금액 계산 (복합 조건용)
이건 왜 있는거야?

아주 날카로운 질문입니다! 단순히 수학적으로 보면 `매수 가격 * 수량` 을 계산한 것뿐이지만, **이 코드가 들어간 이유는 크게 두 가지 전략적 이유**가 있습니다.

결론부터 말씀드리면, **"과제의 난이도와 전문성을 한 단계 높이기 위해"** 넣은 장치입니다.

---

1. 과제 요구사항(and/or 논리 연산자)을 완벽히 수행하기 위함

질문자님의 2차 과제 목표에 "and/or 논리 연산자를 활용하여 상황에 맞는 등급과 특별 칭호를 부여하는 지능형 로직을 완성하라"는 내용이 있었죠?

만약 `invest_amount` 라는 변수가 없다면, 우리는 `profit_rate(수익률)` 하나만 가지고 등급을 나눠야 합니다. 그럼 조건문이 단순해집니다.

- *단순한 조건:* `if profit_rate >= 20:`

하지만 `invest_amount` 가 있으면 **두 가지 조건을 동시에 검사**할 수 있게 됩니다.

- *복합적인 조건:* `if profit_rate >= 20 and invest_amount >= 100000:`

즉, `and` **라는 논리 연산자를 제대로 쓰기 위해 '비교할 대상' 하나를 더 만든 것**입니다. 교수님께 "수익률뿐만 아니라 투자 규모까지 고려한 리스크 관리 로직을 구현하기 위해 이 변수를 추가했다"고 하면, 아주 좋은 점수를 받으실 수 있습니다.

2. '퀀트 전략 매니저'로서의 전문성 (가독성과 의미)

코딩을 할 때 `buy * amt` 를 조건문 안에 직접 쓰는 것보다, `invest_amount = buy * amt` 라고 변수로 빼두면 다음과 같은 장점이 있습니다.

- **가독성:** 코드를 읽는 사람(교수님)이 "아, 학생이 **투자 금액**이라는 변수를 계산해서 쓰고 있구나!"라고 한눈에 파악할 수 있습니다.
- **유지보수:** 만약 나중에 투자 금액 계산 방식이 바뀌더라도(예: 수수료 포함), `invest_amount` 를 구하는 공식 한 줄만 수정하면 아래의 모든 조건문이 자동으로 적용됩니다.

💡 **한 줄 요약**

> **"단순히 수익률만 보는 초보적인 프로그램이 아니라, '투자 규모'라는 또 다른 지표를 활용해 더욱 정교한 퀀트 분석을 하고 있다는 것을 보여주기 위한 장치입니다."**

print(f"종목: {name:10} | 수익률: {profit_rate:6.2f}% | 등급: [{grade}] | 칭호: {title}")
이건 뭐야?

이 코드는 파이썬의 **f-string formatting(포맷팅)** 기법입니다. 쉽게 말해, "출력 결과를 예쁜 표(리포트)처럼 깔끔하게 정리하는 마법"이라고 생각하시면 됩니다.

데이터를 그냥 출력하면 글자 길이에 따라 줄이 삐뚤빼뚤해지는데, 이렇게 코드를 작성하면 **열(column)을 딱딱 맞출 수 있습니다.**

🔍 코드 한 줄씩 뜯어보기

`print(f"종목: {name:10} | 수익률: {profit_rate:6.2f}% | 등급: [{grade}] | 칭호: {title}")`

이 문장을 구성하는 핵심 포인트 2가지입니다.

1. `{name:10}` (문자열 정렬)

- **의미:** `name` 이라는 변수를 출력하되, **무조건 10칸의 공간을 차지하게 하라**는 뜻입니다.
- **효과:** 만약 '삼성전자'가 4글자여도, 뒤에 6칸의 빈 공간을 만들어서, 다음 출력값(수익률)이 항상 일정한 위치에서 시작하게 만듭니다. 덕분에 출력 화면이 **표처럼 정렬**됩니다.

2. `{profit_rate:6.2f}` (숫자 포맷팅)

이 부분은 숫자를 다룰 때 필수입니다.

- `f` : "이 숫자는 실수(float)형이야"라고 알려줍니다.
- `.2` : "소수점 아래 **2자리까지만** 보여줘"라는 뜻입니다. (반올림해서 보여줍니다.)
- `6` : "전체 길이를 **6칸**으로 잡아줘"라는 뜻입니다. (숫자, 소수점, 소수점 아래 숫자들을 포함한 전체 길이입니다.)

이건 무슨 오류야?

분석

이 오류가 발생하는 원인은 아주 명확합니다. 지금 질문자님은 파이썬 파일을 실행하는 '터미널(명령어 창)'에 있는 게 아니라, 파이썬 코드 한 줄 한 줄을 즉석에서 실행하는 '대화형 모드(REPL)'에 들어가 있기 때문입니다.

화면에 보이는 `>>>` 표시가 바로 그 증거입니다.

1. 왜 이런 오류가 나나요?

- **SyntaxError:** 코드 파일 전체를 복사해서 `>>>` 창에 붙여넣으면, 파이썬이 **"들여쓰기(Indentation)"나 "구조(if문, for문)"를 이해하지 못하고 꼬여버립니다.**
- **NameError: name 'o' is not defined:** 아마 코드를 입력하거나 붙여넣으려다가 실수로 키보드 'o'를 누르고 엔터를 치신 것 같습니다. 파이썬은 'o'를 변수 이름이라고 생각했는데, 선언된 적이 없으니 오류를 낸 것입니다.

---

2. 해결 방법 (지금 바로 따라 하세요!)

현재 이 상태를 벗어나서 다시 코드를 실행하려면 아래 순서대로 하세요.

Step 1. 대화형 모드(>>>) 나가기

터미널 창에 아래 명령어 중 하나를 입력하고 엔터를 누르세요.

```
매도가 : 1000
수량 : 1
3번째 기록 :
종목 : 테슬라
매수가 : 500
매도가 : 540
수량 : 5

---수익률 분석 리포트 ---
종목: 최근 이슈 종목  | 수익률:  50.00% | 등급: [S] | 칭호: 시장의 지배자
Traceback (most recent call last):
  File "/workspaces/ie-py-homework-rlawodus441131-cell/main.py", line 29, in <module>
    profit_rate = ((sell - buy) / buy ) * 100
왜 이렇게 나올까
```

이 오류는 코딩할 때 정말 자주 만나는 'ZeroDivisionError (0으로 나누기 오류)'입니다.

🔍 왜 이런 오류가 날까요?

수학에서 **어떤 숫자를 0으로 나누는 것은 불가능**합니다. 파이썬 코드에서 `profit_rate = ((sell - buy) / buy ) * 100` 라고 썼는데, 여기서 `buy` (매수가) 자리에 `0` **이 들어갔기 때문에** 프로그램이 계산을 할 수 없어서 멈춰버린 것입니다.

질문자님이 입력한 데이터나, 리스트에 이미 들어가 있던 데이터 중 **매수가(buy)가 0인 데이터**가 하나라도 있으면 이 오류가 발생합니다.

✅ 해결 방법

`profit_rate` 계산을 하기 직전에, `buy` **가 0인지 확인하고 건너뛰는(skip) 코드**를 반드시 넣어야 합니다. 아래처럼 수정해 보세요.

```
    title = '안정적 트레이더'

    print(f"종목: {name:10} | 수익률: {profit_rate:6.2f}% | 등급: [{grade}] | 칭호: {title}")
print(f"\n총 투자 누적액: {total_sum:,.0f}원")
```

뭐가 문제일까

제공해주신 코드에는 **프로그램이 실행되다가 중간에 멈추거나 오류가 발생하는 치명적인 문제 2가지**가 있습니다. 하나씩 고쳐보겠습니다.

1. 치명적인 오류 1: 리스트 구조 불일치 (런타임 오류 발생)

- **문제:** `trade_history` 를 선언할 때 `['시장지수', 2500, 2600, 1]` 로 만들었습니다. 이렇게 하면 `trade_history` 는 그냥 1차원 리스트입니다.
- **증상:** 나중에 `for trade in analysis_data:` 루프를 돌 때, 첫 번째 값인 `'시장지수'` 를 `name, buy, sell, amount` 라는 4개의 변수에 억지로 넣으려다가 "너무 많은 값을 넣을 수 없다"는 식의 오류가 발생합니다.
- **해결:** 리스트 안에 리스트가 들어있는 **2차원 리스트** 구조로 만들어야 합니다.
  - (수정 전) `trade_history = ['시장지수', 2500, 2600, 1]`
  - (수정 후) `trade_history = [['시장지수', 2500, 2600, 1]]`

2. 치명적인 오류 2: 오타 (NameError 발생)

- **문제:** `else` 문 안에 `tltle = '안정적 트레이더'` 라고 적으셨습니다. `title` 이 아니라 `tltle` 이라고 오타가 났습니다.
- **증상:** 등급이 'B'가 되는 경우에 `title` 변수가 만들어지지 않아서, 마지막 `print` 문에서 `title` 을 찾을 수 없다는 오류가 발생합니다.
- **해결:** `tltle` 을 `title` 로 고쳐주세요.